Phase 1: Database Setup (XAMPP)

1. Start **Apache** and **MySQL** in your XAMPP Control Panel.
2. Open your browser and go to `http://localhost/phpmyadmin`.
3. Click **New** to create a database named `userdb`.
4. Click on the `userdb` database and go to the **SQL** tab.
5. Paste the following SQL and click **Go**:

SQL

```
CREATE TABLE users (
    id INT NOT NULL AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(50),
    country VARCHAR(50),
    PRIMARY KEY (id)
);
```

Phase 2: Project Setup (IntelliJ Community)

1. Open IntelliJ IDEA.
2. Click **New Project**.
3. Select **Maven** from the left sidebar.
4. Check **Create from archetype**.
5. Select `org.apache.maven.archetypes:maven-archetype-webapp`.
6. Click **Next**.
    - **Name:** `UserManagement`
    - **GroupId:** `com.example`
    - **ArtifactId:** `UserManagement`
7. Click **Create/Finish**.

Wait for Maven to download dependencies (check the bottom right progress bar).

Phase 3: Install "Smart Tomcat" Plugin

Since Community Edition removes the Tomcat integration, we add it back via a plugin.

1. In IntelliJ, go to **File** > **Settings** (or `Ctrl+Alt+S`).
2. Go to **Plugins**.
3. Search for **Smart Tomcat**.
4. Click **Install** and then **Restart IDE**.

Phase 4: Configure Dependencies (`pom.xml`)

Open the `pom.xml` file in your project root. You need to add the Servlet API (for Tomcat 9) and the MySQL Connector.

Replace the `<dependencies>` section with this:

XML

```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>javax.servlet-api</artifactId>
        <version>4.0.1</version>
        <scope>provided</scope>
    </dependency>
</dependencies>
```

Important: Look for a floating "M" icon (Maven) in the top right of the code editor and click it to **Load Maven Changes**.

Phase 5: Java Code Implementation

Create a folder structure inside `src/main`:

1. Right-click `src/main`.
2. Create directory: `java`.
3. Right-click `java` -> Mark Directory as -> **Sources Root** (if it's not blue already).
4. Inside `java`, create package `com.example.UserServlet.java`

Phase 6: The Input Form (`index.html`)

Create index.html inside src/main/webapp/.

This is just for adding a new user.

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple CRUD</title>
</head>
<body>
    <h2>Add New User</h2>
    <form action="UserServlet" method="post">
        Name: <input type="text" name="name" required><br><br>
        Email: <input type="text" name="email" required><br><br>
        Country: <input type="text" name="country" required><br><br>
        <input type="submit" value="Save User">
    </form>
    <br>
```

```
    <a href="UserServlet">View All Users</a>
</body>
</html>
```

Phase 7: The Logic (`UserServlet.java`)

Create this file in `src/main/java/com/example/UserServlet.java`.

Java

```
package com.example;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;

@WebServlet("/UserServlet")
public class UserServlet extends HttpServlet {

    // Database Settings
    String url = "jdbc:mysql://localhost:3306/userdb";
    String user = "root";
    String pass = "";

    // 1. POST METHOD: Handles Saving (Insert) AND Updating
    protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        // Get form data
        String id = request.getParameter("id"); // Hidden ID field
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String country = request.getParameter("country");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(url, user, pass);

            if(id == null || id.isEmpty()) {
                // --- CASE 1: CREATE NEW USER (No ID provided) ---
                PreparedStatement ps = con.prepareStatement("INSERT INTO
users (name, email, country) VALUES (?,?,?)");
                ps.setString(1, name);
                ps.setString(2, email);
                ps.setString(3, country);
                ps.executeUpdate();
            } else {
                // --- CASE 2: UPDATE EXISTING USER (ID exists) ---
                PreparedStatement ps = con.prepareStatement("UPDATE users
SET name=?, email=?, country=? WHERE id=?");
                ps.setString(1, name);
                ps.setString(2, email);
                ps.setString(3, country);
                ps.setInt(4, Integer.parseInt(id));
                ps.executeUpdate();
            }
```

```
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }

        // Go back to the list
        response.sendRedirect("UserServlet");
    }

    // 2. GET METHOD: Handles List, Delete, AND Showing the Edit Form
    protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String deleteId = request.getParameter("deleteId");
        String editId = request.getParameter("editId");

        // --- LOGIC A: DELETE USER ---
        if(deleteId != null) {
            deleteUser(deleteId);
            response.sendRedirect("UserServlet"); // Refresh page
            return;
        }

        // --- LOGIC B: SHOW EDIT FORM (Pre-filled) ---
        if(editId != null) {
            showEditForm(editId, out);
            return; // Stop here, don't show the table below
        }

        // --- LOGIC C: SHOW USER LIST (Default) ---
        out.println("<html><body>");
        out.println("<h2>User List</h2>");
        out.println("<a href='index.html'>+ Add New User</a><br><br>");
        out.println("<table border='1' cellpadding='10'>");

out.println("<tr><th>ID</th><th>Name</th><th>Email</th><th>Country</th><th>
Actions</th></tr>");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(url, user, pass);
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM users");

            while (rs.next()) {
                out.println("<tr>");
                out.println("<td>" + rs.getInt("id") + "</td>");
                out.println("<td>" + rs.getString("name") + "</td>");
                out.println("<td>" + rs.getString("email") + "</td>");
                out.println("<td>" + rs.getString("country") + "</td>");

                // Links for Edit and Delete
                int uId = rs.getInt("id");
                out.println("<td>");
                out.println("<a href='UserServlet?editId=" + uId +
"'>Edit</a> | ");
                out.println("<a href='UserServlet?deleteId=" + uId +
"'>Delete</a>");
                out.println("</td>");
```

```
                out.println("</tr>");
            }
            con.close();
        } catch (Exception e) {
            out.println("Error: " + e.getMessage());
        }

        out.println("</table></body></html>");
    }

    // Helper: Generates the HTML Form for Editing
    private void showEditForm(String id, PrintWriter out) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(url, user, pass);
            PreparedStatement ps = con.prepareStatement("SELECT * FROM
users WHERE id=?");
            ps.setString(1, id);
            ResultSet rs = ps.executeQuery();

            if(rs.next()) {
                out.println("<h2>Edit User</h2>");
                out.println("<form action='UserServlet' method='post'>");
                out.println("<input type='hidden' name='id' value='" +
rs.getInt("id") + "'>"); // Hidden ID
                out.println("Name: <input type='text' name='name' value='"
+ rs.getString("name") + "'><br><br>");
                out.println("Email: <input type='text' name='email'
value='" + rs.getString("email") + "'><br><br>");
                out.println("Country: <input type='text' name='country'
value='" + rs.getString("country") + "'><br><br>");
                out.println("<input type='submit' value='Update User'>");
                out.println("</form>");
                out.println("<a href='UserServlet'>Cancel</a>");
            }
            con.close();
        } catch (Exception e) { e.printStackTrace(); }
    }

    // Helper: Deletes User
    private void deleteUser(String id) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(url, user, pass);
            PreparedStatement ps = con.prepareStatement("DELETE FROM users
WHERE id=?");
            ps.setString(1, id);
            ps.executeUpdate();
            con.close();
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Phase 8: Deploy and run project

Step 1: Generate the WAR File (Maven Build)

1. Open your project in **IntelliJ IDEA**.

2. Look at the **Right Sidebar** and click on **Maven** (the 'M' icon).
3. Expand your project name -> **Lifecycle**.
4. **Double-click** on `clean`.
    - *Wait for the console to say "BUILD SUCCESS".*
5. **Double-click** on `install`.
    - *Wait for the console to say "BUILD SUCCESS".*
6. **Find the file:**
    - Look in your **Project Explorer** (Left Sidebar).
    - Go to the `target/` folder.
    - You will see a file named `Nameoffolder.war` (or whatever you named your artifactId). **Remember the location of this file.**

Step 2: Deploy via Browser

1. Open your web browser (Chrome, Edge, etc.).
2. Go to this URL: `http://localhost:8080/manager/html`
3. A login box will appear.
    - **Username:** `admin`
    - **Password:** `password`
    - *(Or whatever you typed in Step 2).*
4. Scroll down to the section **"WAR file to deploy"**.
5. Click **Choose File** (or Browse).
6. Navigate to your IntelliJ project folder -> `target` -> select the `Nameoffolder.war` file.
7. Click the **Deploy** button.

Step 3: Run the Project

1. Look at the list of **Applications** at the top of the Manager page.
2. You will see a new row for `/ Nameoffolder` (or the name of your war file).
3. **Click that link** (e.g., `/ Nameoffolder`).
    - *Note: If your code expects `index.html`, you might need to add it to the URL if it doesn't load automatically.*
    - Example: `http://localhost:8080/Nameoffolder/index.html`